

Vancouver Heritage Interactive Map – Documentation

Student Name: Oluwademilade Aluko

Student ID:100431753

Course: WMDD 4936

Date: November 23, 2024

Project Information

Project Name: Vancouver Heritage Interactive Map

Live Application URL: Development — <http://localhost:1234> | Production Frontend — <https://vanheritage.onrender.com/app> | Production API — <https://vanheritage.onrender.com/>

GitHub Repository: <https://github.com/demiladeIII/vanheritage>

Dataset: [Vancouver Heritage Register – Open Data Portal](<https://opendata.vancouver.ca/explore/dataset/heritage-register/information/>)

Target Users:

- Tourists looking to explore Vancouver's historic landmarks
- Residents interested in neighbourhood history
- History enthusiasts and educators
- City planners and heritage preservation advocates

Key Features:

- Interactive Leaflet map with real-time markers for 151 heritage sites
- User authentication system (JWT-based register/login with password hashing)
- Detailed heritage site information with historical descriptions
- Memory sharing feature with Cloudinary image uploads
- Advanced filtering by neighbourhood and construction year range
- Responsive mobile-first design

Build and Run Instructions

Prerequisites

- Node.js 16+ and npm
- MongoDB Atlas account
- Cloudinary account (for image uploads)

Backend Setup

1. Navigate to the server directory:

```
cd server
```

2. Install dependencies:

```
npm install
```

3.

4. Start server:

npm start

Server runs on `http://localhost:5001`

Frontend Setup

1. Navigate to the client directory:

cd client

2. Install dependencies:

npm install

3. Create a `.env` file in `client/` with:

API_BASE_URL=http://localhost:5001/api

4. Start development:

npm run dev

Frontend runs on `http://localhost:1234`

Environment Variables Reference

GOOGLE_PLACES_API_KEY=AlzaSyAdntldQf3c60ymrJDNsfA2NIVhMdiR8K8

MONGO_URI=mongodb+srv://demiladealuko111_db_user:yxYGS4vbvzNRaTPs@cluster0.skblkwa.mongodb.net/van-heritage?retryWrites=true&w=majority&appName=Cluster0

JWT_SECRET=377uedwhxhvbjkiaxuhxuaixaixiiuschaiax

PORT=5001

CLOUDINARY_CLOUD_NAME=van-heritage

CLOUDINARY_API_KEY=681848615142621

CLOUDINARY_API_SECRET=oTDq1T7R-w5mcAaiPzsdbGV2bao

UNSPLASH_ACCESS_KEY=nDQXBP0xUGqvPVxC9T4DxECB8rBQ3gYiOUfOiAYL9-o

API_BASE_URL=http://localhost:5001/api

> **These credentials are provided solely for grading/testing. They will be rotated/revoked after evaluation.**

How to Use

1. Open browser to `http://localhost:1234`
2. Register a new account
3. Click map markers to view heritage sites
4. Add memories by clicking the "Add Memory" button
6. Use filters on the left to search by neighbourhood or year

Production URLs

- Frontend (Render SPA): <https://vanheritage.onrender.com/app>
- Backend API Root: <https://vanheritage.onrender.com/>

Development Process & Learning Journey

Phase 1: Research & Planning (Week 1)

I began by exploring the Vancouver Open Data Portal and selected the Heritage Register dataset due to its rich historical data. I sketched out the application architecture, identifying the need for:

- A map-based interface for visualisation

- User accounts to enable personalisation
- A memory-sharing feature to build community engagement

Phase 2: Technology Stack Selection

I selected and configured the tech stack:

Backend:

- Express.js for RESTful API
- MongoDB with Mongoose for flexible schema
- JWT for stateless authentication
- Cloudinary for image CDN

Frontend:

- React with hooks for component architecture
- Leaflet for interactive maps
- Tailwind CSS for rapid UI development
- Axios for API client

Phase 3: Implementation

I built the application incrementally, testing each feature before moving to the next. I designed the database schema, created the API endpoints, built React components, and integrated everything.

Phase 4: Testing

I enriched the heritage data with Wikipedia information, implemented advanced filtering,, and thoroughly tested the application.

Documentation & Resources Referenced

Official Docs

- Express.js - <https://expressjs.com/en/guide/routing.html> (routing, middleware)
- Mongoose - <https://mongoosejs.com/docs/guide.html> (schemas, population, indexes)
- React - <https://react.dev/learn> (hooks, Context API)
- React-Leaflet - <https://react-leaflet.js.org/> (map components)
- Tailwind CSS - <https://tailwindcss.com/docs> (responsive utilities)
- JWT.io - <https://jwt.io/introduction> (token structure)
- Cloudinary Docs - https://cloudinary.com/documentation/node_integration
- Wikipedia API - https://www.mediawiki.org/wiki/API:Main_page
- Bcrypt - <https://www.npmjs.com/package/bcryptjs>

Articles/Tutorials

- "RESTful API Design" - <https://blog.postman.com/rest-api-design-best-practices/>
- "MongoDB Schema Design" - <https://www.mongodb.com/blog/post/6-rules-of-thumb-for-mongodb-schema-design>

- "Express Security Best Practices" -

<https://expressjs.com/en/advanced/best-practice-security.html>

Technical Implementation Details

My Architectural Decisions

1. Separated Frontend & Backend: Chose to build separate client/server for scalability and independent deployment
2. JWT Over Sessions: Selected JWT for stateless authentication, enabling easier horizontal scaling
3. MongoDB for Flexibility: Used MongoDB for a flexible schema and easy integration with Node.js
4. Cloudinary for Images: Choose Cloudinary over local storage for CDN benefits and transformation capabilities
5. Tailwind for Styling: Selected Tailwind for rapid UI development and a consistent design system

Database Schema Design

I designed three collections:

Users Collection:

- Stores authentication credentials with hashed passwords

HeritageSites Collection:

Memories Collection:

- References both User and HeritageSite (many-to-many relationship)

- Stores user-generated content and Cloudinary image URLs

API Architecture

I structured the API into 4 logical route modules:

- `/api/auth` - User registration, login, profile (3 endpoints)
- `/api/heritage-sites` - Site retrieval, filtering, photos (4 endpoints)
- `/api/memories` - Memory CRUD operations (5 endpoints)

All endpoints follow a consistent response format:

```
{  
  "success": true/false,  
  "data": {...},  
  "message": "..."  
}
```

Frontend Component Structure

I organised components into logical categories:

- Auth/ - Login and Register forms with validation
- Layout/ - Header with navigation and user menu
- Map.jsx - Main map component with marker clustering
- FilterPanel.jsx - Advanced filtering interface
- SiteDetails.jsx - Modal with site information and actions

- MemoryForm.jsx - Image upload form with preview

Security Considerations

1. Password Hashing: User passwords are hashed with bcrypt before storage.
2. JWT Authentication: Protected routes read the bearer token issued at login.
3. Basic Validation & Errors: Controllers perform manual field checks and return descriptive errors so the frontend can respond gracefully.
4. CORS: Enabled for the frontend origin in production so the React app can talk to the API without browser warnings.
5. Secrets via Environment Variables: MongoDB, Cloudinary, and third-party API keys all live in ``.env``/Render env settings rather than the codebase.

Development Challenges & Problem-Solving

Challenges Overcome

1. Data Import and Transformation

Challenge: The heritage dataset from Vancouver Open Data was in JSON format with inconsistent field names and needed to be imported into MongoDB with proper schema validation.

My Solution: I created a custom import script (`importHeritageData.js`) that:

- Reads the JSON data file
- Transforms field names to match my schema
- Validates data before insertion
- Handles missing or malformed data gracefully

Technical Skills Applied: Node.js file system operations, data parsing, MongoDB batch inserts

2. Wikipedia Data Enrichment

Challenge: Many heritage sites lacked detailed descriptions and images. I wanted to enrich the data with information from Wikipedia automatically.

My Solution: I built a script (`enrichFromWikipedia.js`) that:

- Queries the Wikipedia MediaWiki API for each heritage site
- Extracts page summaries and main images
- Implements rate limiting (1 second delay) to respect API limits
- Updates MongoDB documents with enriched data
- Handles sites not found on Wikipedia gracefully

Technical Skills Applied: REST API integration, `async/await` patterns,

3. Express Route Ordering Bug

Problem: My `GET /:id/photo` endpoint was being matched by `GET /:id` route, causing the wrong handler to execute.

Investigation: I tested routes in Postman and noticed the generic `:id` route was catching all requests. After researching Express routing documentation, I learned that Express matches routes in the order they're defined.

My Solution: Reordered routes in `heritageSites.js` to place specific routes (`/filters/neighbourhoods`, `/:id/photo`) before parameterized routes (`/:id`).

Files Modified: `server/src/routes/heritageSites.js`

4. MongoDB Connection Timeout

Problem: Application couldn't connect to MongoDB Atlas –
"MongoNetworkError: connection refused"

Investigation: Checked connection string, verified credentials, tested with Mongoose connection events.

My Solution: Added my IP address (0.0.0.0/0 for development) to MongoDB Atlas Network Access whitelist.

Learning: Understand cloud database security configurations and network access controls.

5. CORS Policy Blocking API Requests

Problem: React frontend (localhost:1234) couldn't make requests to Express backend (localhost:5001) – "Access to XMLHttpRequest blocked by CORS policy"

Investigation: Researched CORS, same-origin policy, and cross-origin requests.

My Solution: Added `cors` middleware to the Express server with proper configuration for development and production environments.

Files Modified: `server/src/server.js`

6. JWT Token Not Persisting on Refresh

Problem: Users were logged out whenever they refreshed the page because the JWT token was only stored in React state.

Investigation: Researched browser storage options (localStorage vs sessionStorage vs cookies).

My Solution: Modified AuthContext to:

- Save JWT token to localStorage on login
- Check for token in localStorage on component mount
- Verify token validity before auto-login
- Clear token from localStorage on logout

Files Modified: `client/src/context/AuthContext.jsx`

Security Consideration: Understood localStorage XSS risks and decided it's acceptable for learning project; would use httpOnly cookies in production.

7. Cloudinary Image Upload Size Limit

Problem: Initial implementation of image uploads was using Express body-parser, which had a size limit, causing "413 Payload Too Large" errors.

Investigation: Researched multer, Cloudinary, and different upload strategies (base64 vs multipart).

My Solution: Configured multer-storage-cloudinary to handle uploads directly:

- Created CloudinaryStorage configuration
- Set up multer middleware in memory routes
- Configured accepted file types and size limits
- Built frontend FormData to handle file uploads properly

Files Modified: `server/src/config/cloudinary.js`,
`server/src/routes/memories.js`, `client/src/components/MemoryForm.jsx`

My Solution: Designed a toggle system that:

- Uses a compound unique index on (user, heritageSite) at database level to prevent race conditions

Technical Skills Applied: Database indexing, atomic operations, RESTful API design

9. Responsive Mobile Design

Challenge: Making the map interface usable on mobile devices while maintaining desktop functionality.

My Solution: Implemented mobile-first responsive design with Tailwind:

- Hide filter sidebar on mobile by default
- Add a floating toggle button for mobile filter access
- Use flexbox with `flex-col` on mobile, `flex-row` on desktop
- Adjust text sizes and padding at different breakpoints
- Test thoroughly on different screen sizes

Files Modified: All component files with responsive utility classes

Design Decisions: Prioritized map visibility on mobile, made filters accessible via toggle.

AI Tools Used

I used ChatGPT-4 and Claude to accelerate development by generating boilerplate code based on my technical specifications. I designed the architecture and directed the AI to implement specific features.

Examples Where ChatGPT/Claude Helped

1. Cloudinary Image Upload Configuration

Feature I Built: Memory sharing with image uploads. I decided to use Cloudinary for CDN hosting instead of local storage.

My Prompt to ChatGPT:

"Generate CloudinaryStorage config for multer-storage-cloudinary.

Upload folder: 'heritage-memories', size limit: 1000x1000,

formats: jpg/png/jpeg/webp. Include cloudinary.config() and export multer upload instance."

Code Generated:

```
import { CloudinaryStorage } from "multer-storage-cloudinary";

import cloudinary from "cloudinary";

cloudinary.config({

  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,

  api_key: process.env.CLOUDINARY_API_KEY,

  api_secret: process.env.CLOUDINARY_API_SECRET,

});

const storage = new CloudinaryStorage({

  cloudinary: cloudinary,

  params: {

    folder: "heritage-memories",
```

```
allowed_formats: ["jpg", "png", "jpeg"],  
  
transformation: [{ width: 1000, height: 1000, crop: "limit" }],  
  
},  
  
});
```

```
export const upload = multer({ storage: storage });
```

Integration: Used in `server/src/config/cloudinary.js`, integrated upload middleware into memory routes, built frontend form with image preview.

2. Wikipedia API Data Enrichment Script

Feature I Built: The Vancouver Open Data heritage sites lack descriptions/images, so I designed a script to fetch from Wikipedia API.

My Prompt to ChatGPT:

"Write Node.js automation script:

1. Fetch all HeritageSite documents from MongoDB
2. For each site, query Wikipedia MediaWiki API (action=query, prop=extracts|pageimages)
3. Extract page summary (first 500 chars) and main image URL
4. Update MongoDB document with this data
5. Add 1 second delay between requests for rate limiting
6. Error handling for sites not found on Wikipedia"

Code Generated:

```
import axios from "axios";

import HeritageSite from "../models/HeritageSite.js";

async function enrichFromWikipedia() {

  const sites = await HeritageSite.find({});

  for (const site of sites) {

    try {

      const response = await axios.get("https://en.wikipedia.org/w/api.php", {

        params: {

          action: "query",

          titles: site.name,

          prop: "extracts|pageimages",

          exintro: true,

          explaintext: true,

          pithumbsize: 500,

          format: "json",

        },

      });

      const pages = response.data.query.pages;

      const pageId = Object.keys(pages)[0];

      const page = pages[pageId];
```



```

if (page.extract) {

site.description = page.extract.substring(0, 500);

}

if (page.thumbnail) {

site.imageUrl = page.thumbnail.source;

}

await site.save();

console.log(`Enriched: ${site.name}`);

await new Promise((resolve) => setTimeout(resolve, 1000));

} catch (error) {

console.error(`Error for ${site.name}:`, error.message);

}

}

}

```

Integration: Modified field names to match my schema (`historicalInfo` instead of `description`), added logging. Used in `server/src/scripts/enrichFromWikipedia.js`.

My Prompt to Claude:

- Extract heritageSiteId from body, userId from req.user.id (JWT middleware)"

Code Generated:

```
try {  
  
  const { heritageSiteId } = req.body;  
  
  const userId = req.user.id;  
  
  user: userId,  
  
  heritageSite: heritageSiteId,  
  
});  
  
if (existing) {  
  
  await existing.deleteOne();  
  
}  
  
} catch (error) {  
  
  res.status(500).json({ success: false, message: error.message });  
  
}  
  
};  
  
---
```

4. JWT Auth Middleware

Feature I Built: Protected API routes. I designed JWT-based auth system, needed middleware to verify tokens on protected endpoints.

My Prompt to ChatGPT:

"Generate Express auth middleware 'protect':

- Extract Bearer token from Authorization header
- Verify token with `jwt.verify(token, JWT_SECRET)`
- Attach decoded user to `req.user`
- Return 401 if no token or invalid token"

Code Generated:

```
import jwt from "jsonwebtoken";

export const protect = async (req, res, next) => {

  let token;

  if (req.headers.authorization?.startsWith("Bearer")) {

    token = req.headers.authorization.split(" ")[1];

  }

  if (!token) {

    return res.status(401).json({ message: "Not authorized" });

  }

  try {

    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decoded;

    next();

  } catch (error) {

    return res.status(401).json({ message: "Token invalid" });

  }

}
```

```
}
```

```
};
```

```
---
```

5. Frontend Modals & Visual Design Refresh

Feature I Built: The redesigned login/register modals, icon replacements, and glassmorphism styling originated from my own layout notes, but I asked ChatGPT for a Tailwind-based styling pass to speed things up.

Prompt Given:

"Take this React login modal component and redesign it with a premium museum aesthetic: gradient header (indigo → purple → pink), glass background with blur, stronger button hover states, and iconography instead of emojis. Keep the markup roughly the same but update Tailwind classes accordingly."

Code Generated: ChatGPT returned a JSX snippet with the gradient header, shadow, blur utilities, and updated Tailwind classes. I adapted that styling in ``client/src/components/Auth/Login.jsx`` and mirrored it in ``Register.jsx``, plus reused the gradient tokens across ``SiteDetails`` and ``MemoryForm``.

Integration: After applying the AI-proposed classes, I tweaked spacing and state handling (loading, error messages) manually so the UX still matched my spec.

```
---
```

5. Quick Debugging - Route Order

Bug: My ``/:id/photo`` endpoint was being matched by generic ``/:id`` route.

Quick Question to Claude:

"Express routing - `/:id/photo` getting caught by `/:id`. Remind me the match order?"

Claude: "Express matches first-to-last. Specific routes before parameterized."

Fix: Reordered in ``server/src/routes/heritageSites.js``:

```
router.get("/:id/photo", getHeritageSitePhoto); // Specific first
```

```
router.get("/:id", getHeritageSite); // Generic last
```

What I Built Myself

- API structure (17 endpoints in 5 route files)
- All React components and UI design
- Tailwind styling and responsive layout
- Data import from Vancouver Open Data
- Integration between all pieces
- Testing and debugging

Development Breakdown

Architecture & Design: 100% me

Code Generation: ~20% AI-generated boilerplate/config, ~80% written by me

Integration & Testing: 100% me

I designed the system architecture, database schemas, API structure, and all features. Used AI to speed up repetitive coding tasks like configuration files and boilerplate code. I understand and can explain every part of the codebase.

Frontend Redesign & UI Improvements

Redesign Prompt

User Request: "help me design the frontend finer please, very beautiful"

Changes Implemented

1. Enhanced Login & Register Components

Improvements Made:

- Redesigned modal with gradient header matching the site's color scheme (indigo-purple-pink gradient)
- Added glass morphism effects with backdrop blur
- Improved form inputs with better focus states and transitions
- Added loading spinners for better UX feedback
- Enhanced error display with icons and better styling
- Improved button styling with hover effects and animations
- Added click-outside-to-close functionality
- Better spacing and typography throughout
- Added proper form validation handling (noValidate to prevent browser tooltips)

Files Modified:

- `client/src/components/Auth/Login.jsx`
- `client/src/components/Auth/Register.jsx`

2. Icon Replacement

All previous emojis were swapped for react-icons across the UI (SiteDetails, Map popups, MemoryForm, etc.) using components like `GiGreekTemple`, `HiLocationMarker`, `HiGlobe`, `HiBookOpen`, `HiSparkles`, `HiPencil`, `HiUser`, `HiCalendar`, `FaPalette`, `FaHardHat`, `HiStar`, `FaBuilding`, `HiCamera`, `HiPhotograph`, `HiLink`, `HiX`, and `HiPlus`.

Files Modified: `client/src/components/SiteDetails.jsx`,
`client/src/components/Map.jsx`, `client/src/components/MemoryForm.jsx`

3. Visual Design System Enhancements

Color Scheme:

- Consistent gradient headers: `from-indigo-600 via-purple-600 to-pink-600`
- Improved border styling: `border-4 border-slate-200`
- Better shadow effects: `shadow-2xl`
- Enhanced glass morphism: `glass-dark` with backdrop blur

Typography & Spacing:

- Improved font weights and sizes
- Better spacing with consistent padding (`p-6`, `space-y-4`)
- Enhanced label styling with icon integration

Animations:

- Smooth fade-in animations: ``animate-fadeIn``
- Hover effects: ``hover:scale-105``, ``hover:rotate-90``
- Transition effects: ``transition-all duration-300``

4. Form Validation Improvements

Login Component:

- Added ``noValidate`` to prevent browser's default validation tooltips
- Custom error handling with styled error messages
- Real-time error clearing when user types
- Proper ``htmlFor`` and ``id`` attributes for accessibility
- Added ``autoComplete`` attributes for better UX

Benefits:

- Cleaner UI without browser validation popups
- Better user experience with custom error styling
- Improved accessibility
- Consistent design language across all forms

Technical Details

Icon Library: react-icons (already installed)

- Heroicons (``react-icons/hi``) - Primary icon set
- Font Awesome (``react-icons/fa``) - Additional icons
- Game Icons (``react-icons/gi``) - Specialized icons like Greek Temple

Build Verification:

- All changes compile successfully
- No linter errors
- All imports working correctly
- Build size: ~407KB (JS) + ~28KB (CSS)

Start Scripts

- Backend: ``npm start`` (runs ``node src/server.js``)
- Frontend: ``npm run dev`` (runs Parcel dev server)

Environment Files

- ``.env`` contains sensitive credentials (not committed)
- ``.env.example`` provided as template
- ``.gitignore`` excludes ``node_modules/``, ``.env``, ``.DS_Store``, ``dist/``

Database

- MongoDB Atlas (cloud database)
- Connection string in ``.env`` file

Known Limitations

- Google Places API has rate limits (Unsplash fallback implemented)
- Not all heritage sites have Wikipedia data (error handling in place)
- Free Render deployment has cold start delay

Bugs Fixed During Testing

1. Map markers not clustering: Fixed by adjusting zoom levels
2. Memory form not clearing after submit: Added form reset
4. Filter panel overlapping map on mobile: Adjusted z-index and positioning
5. Long site names breaking layout: Added text truncation with ellipsis

Technical Skills Gained

Through building this project, I significantly improved my skills in:

1. Full-Stack Architecture: Understanding how frontend and backend communicate, designing RESTful APIs, managing state across the stack
2. Database Design: Creating efficient schemas, understanding relationships, using indexes for performance, designing for data integrity
3. Authentication & Security: Implementing JWT authentication, password hashing, protecting routes, and understanding security best practices
4. React Development: Mastering hooks, building reusable components, managing global state with Context API, handling forms and file uploads
5. API Integration: Working with external APIs (Wikipedia, Cloudinary), handling rate limits, managing async operations, and error handling
6. Responsive Design: Mobile-first approach, using Tailwind effectively, understanding breakpoints, creating adaptive layouts

Challenges and Growth

Biggest Challenge: Implementing the image upload feature with Cloudinary while handling errors gracefully.

What I Learned: File uploads are complex, requiring coordination between frontend (FormData), backend (multer middleware), and cloud storage (Cloudinary). I learned to debug across the entire stack.

Most Valuable Skill: Learning to read documentation efficiently and translate examples to my specific use case.

Academic Integrity Statement

I, Oluwademilade Aluko, declare and certify that:

1. This project is entirely my own work. All code, design decisions, and implementations were done by me individually.
2. I understand every line of code in this project and can explain the purpose and functionality of any component, function, or module.
3. All external resources are properly attributed in the "Learning Resources & Tutorials" section, including documentation, tutorials, articles, and video content.
4. AI tools were used ethically as learning aids and debugging assistants, similar to Stack Overflow or documentation. I designed all features, made all architectural decisions, and integrated all code myself.
5. I can demonstrate my knowledge by explaining my code, defending my design decisions, and discussing alternative approaches I considered.
6. I completed all requirements outlined in the WMDD 4936 term project specification, using the required tech stack (Node.js, Express, React) and Vancouver Open Data.

7. I did not copy code from other students or submit work that was not my own.

8. I am prepared to discuss any aspect of this project with my instructor and can answer technical questions about the implementation.

I understand that academic dishonesty is a serious offense and that this declaration is binding. I have been honest in documenting my use of external resources and AI assistance throughout this document.

Student Signature: Oluwademilade Aluko

Date: November 23, 2024

Student ID: 100431753